Module 3

This is a single, concatenated file, suitable for printing or saving as a PDF for offline viewing. Please note that some animations or images may not work.

Module 3 Study Guide and Deliverables

Readings:	Brown: Chapters 13–17
Assignments:	Assignment 3 due Tuesday, February 7 at 6:00 AM ET
Live Classroom:	• Tuesday, January 31, 6:00–8:00 PM ET • Facilitator live office: TBD

Persistence using MySQL

Learning Objectives

By reading the lectures and textbook, participating in the discussions, and completing the assignments, you will be able to do the following:

- Write standalone MySQL database applications.
- Develop Node web applications that persist and interact with data form MySQL database.

Introduction

MySQL is one of the popular relational open source databases. The instructions for installing MySQL can be found on the MySQL web site.

The community edition of MySQL is freely available and is used in this lecture. A default database named, *test*, is provided by the installation. The examples use the credentials *root* and the empty string for the username and password, respectively.

For the *Node* applications to interact with the MySQL database, the *mysql* module is required and is installed for the applications as shown below. This is the *Node.js* driver for MySQL written in JavaScript.

```
>npm install --save mysql
```

Connecting to MySQL

A connection is required for accessing the database. The connection object is created using the createConnection() method from the *mysql* module. The method takes an object as its argument with the properties shown below.

ex01_mysql.js

```
var mysql = require("mysql");
var credentials = require("./credentials.js");

var connection = mysql.createConnection({
  "host": "localhost", "port": 3306,
  "user": credentials.username,
  "password": credentials.password,
  "database": "test"
});
```

Using the connection object, the connection to the database is established using the connect() method. The argument for this method is the callback function that is invoked after the connection is established. After the database operations are completed, the connection can be closed using the end() method. This method closes the connection gracefully by letting any queued queries to execute completely. An alternate approach is to use the destroy() method. This method immediately closes the database connection. The following example illustrates the typical usage of the connect() and end() methods.

ex01_mysql.js

```

connection.connect(function(error) {
  if (error) {
    console.error(error);
    return;
  }
  // Connection successfully established
  console.log("Connected...");
});

// Do database operations

connection.end(function(error) {
  if (error) {
    console.error(error);
    return;
  }
  // Connection successfully closed
  console.log("Closed...");
});
```

Executing the above program results in the following output in the successful scenario.

```
>node ex01_mysql.js
Connected...
Closed...
```

If the MySQL instance is not running, the above program results in an error while trying to establish a connection, as shown below.

```
>node ex01_mysql.js
{ [Error: connect ECONNREFUSED]
  code: 'ECONNREFUSED',
  errno: 'ECONNREFUSED',
  syscall: 'connect',
  fatal: true }
```

If incorrect credentials are used when MySQL instance is running, the above program results in an error while trying to establish a connection, as shown below.

```
>node ex01_mysql.js
{ [Error: ER_ACCESS_DENIED_ERROR: Access denied for user
'root'@'localhost' (using password: YES)]
  code: 'ER_ACCESS_DENIED_ERROR',
  errno: 1045,
  sqlState: '28000',
  fatal: true }
```

Executing Database Queries – SELECT Statement

The following schema and initial data shown is used for demonstrating the execution of SQL queries.

createdb.sql

```
1  drop table if exists Weather;
2  create table Weather (
3    city varchar(100) not null,
4    temperature integer not null
5  );
6
7  insert into Weather (city, temperature) values ('Austin', 48);
8  insert into Weather (city, temperature) values ('Baton Rouge', 57);
9  insert into Weather (city, temperature) values ('Jackson', 50);
10 insert into Weather (city, temperature) values ('Montgomery', 53);
11 insert into Weather (city, temperature) values ('Phoenix', 67);
12 insert into Weather (city, temperature) values ('Sacramento', 66);
13 insert into Weather (city, temperature) values ('Santa Fe', 27);
14 insert into Weather (city, temperature) values ('Tallahassee', 59);
```

For executing SQL queries, after connecting to the database, the connection object's query() method is used. The two arguments for this method are the SQL string to execute and the callback function. After the query is executed, the callback function is invoked with the possible results or an error.

The following example shows the selection of all the rows from the *Weather* table. On successful completion, the results are available as an array of objects. Each object contains the information for each matched row of the query. The selected columns from the table are mapped as properties of these objects.

ex02_mysql.js

```
19
20  // Do database operations
21
22  connection.query("Select * from Weather",
23    function(error, rows) {
24      if (error) {
25        console.error(error);
26        return;
27      }
28      console.log(rows);
29    });
```

Executing the above program results in the following output in the successful scenario.

```
>node ex02_mysql.js
Connected...
[ { city: 'Austin', temperature: 48 },
  { city: 'Baton Rouge', temperature: 57 },
  { city: 'Jackson', temperature: 50 },
  { city: 'Montgomery', temperature: 53 },
  { city: 'Phoenix', temperature: 67 },
  { city: 'Sacramento', temperature: 66 },
  { city: 'Santa Fe', temperature: 27 },
  { city: 'Tallahassee', temperature: 59 } ]
Closed...
```

The following example iterates over the results, accesses the *city* and *temperature* properties for each result, and prints a custom output to the console.

ex03_mysql.js

```
20  // Do database operations
21
22  connection.query("Select * from Weather",
23    function(error, rows) {
24      if (error) {
25        console.error(error);
26        return;
27      }
28      for (var i = 0; i < rows.length; i++) {
29        console.log(rows[i].city, "-",
30                    rows[i].temperature + " degrees");
31      }
32    });
```

The output from the above program is shown below.

```
>node ex03_mysql.js
Connected...
Austin - 48 degrees
Baton Rouge - 57 degrees
Jackson - 50 degrees
Montgomery - 53 degrees
Phoenix - 67 degrees
Sacramento - 66 degrees
Santa Fe - 27 degrees
Tallahassee - 59 degrees
Closed...
```

Executing Database Queries - INSERT Statement

For inserting new data into the table in the database, the INSERT prepared statement is used. If the data is available as object, the placeholder in the query is used to set the specified data as shown in the following example.

ex04_mysql.js

```
20  // Do database operations
21
22  var newData = {city: 'Oklahoma City', temperature: 50};
23
24  connection.query("Insert into Weather SET ?", newData,
25    function(error, result) {
26      if (error) {
27        console.error(error);
28        return;
29      }
30      console.log(result);
31    });
```

If no error occurs, the callback function's result is an object with the structure as shown below. The *affectedRows* property shows the number of rows affected in the database as a result of this operation.

```
>node ex04_mysql.js
Connected...
{ fieldCount: 0,
  affectedRows: 1,
  insertId: 0,
  serverStatus: 2,
  warningCount: 0,
  message: '',
  protocol41: true,
  changedRows: 0 }
Closed...
```

Executing the program selecting all the rows shows the new data that got inserted into the table as illustrated below.

```
>node ex03_mysql.js
Connected...
Austin - 48 degrees
Baton Rouge - 57 degrees
Jackson - 50 degrees
Montgomery - 53 degrees
Phoenix - 67 degrees
Sacramento - 66 degrees
Santa Fe - 27 degrees
Tallahassee - 59 degrees
Oklahoma City - 50 degrees
Closed...
```

Executing Database Queries - UPDATE Statement

For updating the current data in the database, the UPDATEprepared statement is used with placeholders. If the data is available as an array, the placeholders in the query are used to set the specified data as shown in the following example. The order of the data in the array must match the order of the placeholders in the query.

ex05_mysql.js

```
20  // Do database operations
21
22  var newData = [60, 'Oklahoma City'];
23
24  connection.query(
25    "Update Weather SET temperature = ? Where city = ?",
26    newData,
27    function(error, result) {
28      if (error) {
29        console.error(error);
30        return;
31      }
32      console.log(result);
33    });
```

If no error occurs, the callback function's result is an object with the structure as shown below. The *affectedRows* and *changedRows* properties show the number of rows affected in the database as a result of this operation.

```
>node ex05_mysql.js
Connected...
{ fieldCount: 0,
  affectedRows: 1,
  insertId: 0,
  serverStatus: 34,
  warningCount: 0,
  message: '(Rows matched: 1  Changed: 1  Warnings: 0',
  protocol41: true,
  changedRows: 1 }
Closed...
```

Executing the program selecting all the rows shows the updated from the table as illustrated below.

```
>node ex03_mysql.js
Connected...
Austin - 48 degrees
Baton Rouge - 57 degrees
Jackson - 50 degrees
Montgomery - 53 degrees
Phoenix - 67 degrees
Sacramento - 66 degrees
Santa Fe - 27 degrees
Tallahassee - 59 degrees
Oklahoma City - 60 degrees
Closed...
```

Executing Database Queries – DELETE Statement

For deleting specific data in the database, the DELETEprepared statement is used with placeholders, if necessary. If the data is available as an array, the placeholders in the query are used to set the specified data as shown in the following example. The order of the data in the array must match the order of the placeholders in the query, if more than one field is needed in the query.

ex06_mysql.js

```
// Do database operations

var newData = ['Oklahoma City'];

connection.query(
  "Delete from Weather Where city = ?",
  newData,
  function(error, result) {
    if (error) {
      console.error(error);
      return;
    }
    console.log(result);
  });
```

If no error occurs, the callback function's result is an object with the structure as shown below. The *affectedRows* property shows the number of rows affected in the database as a result of this operation.

```
>node ex06_mysql.js
Connected...
{ fieldCount: 0,
  affectedRows: 1,
  insertId: 0,
  serverStatus: 34,
  warningCount: 0,
  message: '',
  protocol41: true,
  changedRows: 0 }
Closed...
```

Executing the program selecting all the rows shows the updated from the table as illustrated below.

```
>node ex03_mysql.js
Connected...
Austin - 48 degrees
Baton Rouge - 57 degrees
Jackson - 50 degrees
Montgomery - 53 degrees
Phoenix - 67 degrees
Sacramento - 66 degrees
Santa Fe - 27 degrees
Tallahassee - 59 degrees
Closed...
```

Transaction support is available through the connection object's beginTransaction(), commit(), and rollback() methods.

Connection Pooling

If frequent connections are required to the database, creating a database connection pool is more efficient. The connection pool object is created using the createPool() method from the *mysql* module. The method takes an object as its argument with the properties shown below.

ex07_mysql.js

```
var mysql = require("mysql");
var credentials = require("./credentials.js");

var pool = mysql.createPool({
  "host": "localhost", "port": 3306,
  "user": credentials.username,
  "password": credentials.password,
  "database": "test"
});

```

Each time a new connection is required, the connection is obtained from the pool. Once the operations are done, the connection can be returned to the pool. The createPool() method also supports additional options for the connectionLimit and queueLimit. The connectionLimit property defaults to 10 and specifies the maximum number of connections that can be created at once. The queueLimit property specifies the maximum number of connection requests that can be queued by the pool. The default value is 0, indicating no limit.

The connection pool's getConnection() method is used for requesting a database connection. The callback function specified as its argument provides the requested connection object, as shown below. The release() method on the connection object returns the connection to the pool.

ex07_mysql.js

```
pool.getConnection(function(error, connection) {
  if (error) {
    console.error(error);
    return;
  }
  // Connection successfully established
  console.log("Connection from pool...");

  // Do database operations

  connection.query("Select * from Weather",
  function(error, rows) {
    if (error) {
      console.error(error);
      return;
    }
    console.log(rows);
  });

  // Release the connection to the pool
  connection.release();

});
```

Executing the program selecting all the rows shows the data from the table as illustrated below.

```
>node ex07_mysql.js
Connection from pool...
[ { city: 'Austin', temperature: 48 },
  { city: 'Baton Rouge', temperature: 57 },
  { city: 'Jackson', temperature: 50 },
  { city: 'Montgomery', temperature: 53 },
  { city: 'Phoenix', temperature: 67 },
  { city: 'Sacramento', temperature: 66 },
  { city: 'Santa Fe', temperature: 27 },
  { city: 'Tallahassee', temperature: 59 } ]
```

Escaping User Input

In order to avoid the SQL Injection attacks, care should be taken while handling any input data provided by the user. Any user provided data should be escaped using the mysql.escape(), connection.escape(), or pool.escape() methods as shown below.

```
var input = 'Austin';
connection.query("Select * from Weather WHERE city = " +
                 connection.escape(input),
  function(error, rows) {
    if (error) {
      console.error(error);
      return;
    }
    console.log(rows);
  });
```

The alternative is to use ? characters as placeholders in the query as shown below.

```
var input = 'Austin';
connection.query("Select * from Weather WHERE city = ?",
                 [input],
  function(error, rows) {
    if (error) {
      console.error(error);
      return;
    }
    console.log(rows);
  });
```

Case Study - Course Manager

The following example shows the web application to manage the list of courses. The schema and the initial data for the database are shown below.

courses.sql

```sql
drop table if exists met_courses;

create table met_courses (
  id int(11) NOT NULL AUTO_INCREMENT,
  course_number varchar(32) NOT NULL,
  course_name   varchar(200) NOT NULL,
  PRIMARY KEY (id)
);

INSERT INTO met_courses (id, course_number, course_name) VALUES
    (1, 'cs520', 'Information Structures'),
    (2, 'cs601', 'Web Application Development'),
    (3, 'cs602 ', 'Server-Side Web Development');
```

The web application provides the capability for displaying all the courses, edit or delete an existing course, and adding a new course. The web application uses Express and Express Handlebars as the view template engine. The Express Router is used for specifying the end points and their handlers, as shown below.

ex08_caseStudy.js

```js
var express = require('express');
var bodyParser = require('body-parser');
var handlebars = require('express-handlebars');

var app = express();

// setup handlebars view engine
app.engine('handlebars',
    handlebars({defaultLayout: 'main_logo'}));
app.set('view engine', 'handlebars');

// static resources
app.use(express.static(__dirname + '/public'));

app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: false }));

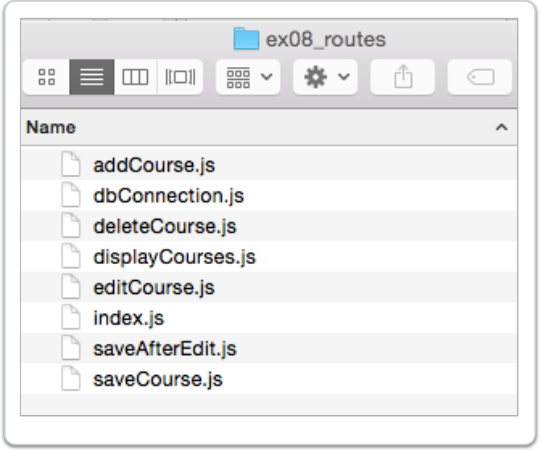
// Routing
var routes = require('./ex08_routes/index');
app.use('/', routes);
```

The routes are configured for the paths as illustrated in the following table:

Route	HTTP	Function
/courses	GET	Display all the courses from the database
/courses/add	GET	Show form to add a new course
/courses/add	POST	Save the new course to the database

/courses/edit/:id	GET	Show form with the data about the course with the specified id
/courses/edit/:id	POST	Update the course data in the database for the specified id
/courses/delete/:id	GET	Delete the course with the specified id from the database

Each of the above operations is specified in their own modules as shown in the following structure.



The *index.js* file creates the router with the routes and their handlers as shown below. The module exported by this file is used in the main web application.

index.js

```
1  var express = require('express');
2  var router = express.Router();
3
4  // other modules
5  var displayCourses  = require("./displayCourses");
6  var addCourse       = require("./addCourse");
7  var saveCourse      = require("./saveCourse");
8  var editCourse      = require("./editCourse");
9  var saveAfterEdit   = require("./saveAfterEdit");
10 var deleteCourse    = require("./deleteCourse");
11
12 // router specs
13 router.get('/', function(req, res, next) {
14   res.redirect('/courses');
15 });
16
17 router.get('/courses',             displayCourses);
18
19 router.get('/courses/add',         addCourse);
20 router.post('/courses/add',        saveCourse);
21
22 router.get('/courses/edit/:id',    editCourse);
23 router.post('/courses/edit/:id',   saveAfterEdit);
24
25 router.get('/courses/delete/:id', deleteCourse);
26
27 module.exports = router;
```

The *displayCourses* module exports the function that retrieves all the data from the database table passes the data to the view to be rendered as shown below.

displayCourses.js

```
var connection =
  require('./dbConnection.js').dbConnect();

module.exports =
  function displayCourses(req , res , next){
    connection.query('select * from met_courses',
      function(err , rows){
        if(err)
            console.log("Error Selecting : %s ",err);
        res.render('displayCoursesView',
          {title:"List of Courses", data:rows});
    });
};
```

The view page for displaying all the course data is shown below. In addition to the course name and course number, links are provided for editing and deleting each course. The view also has the button for adding a new course.

displayCoursesView.handlebars

```
<div>
  <div class="table-responsive">
    <table class="table table-condensed">
      <thead>
        <tr>
          <th>Course Number</th>
          <th>Course Name</th>
          <th></th>
        </tr>
      </thead>
      <tbody>
        {{#each data}}
          <tr>
            <td>{{this.course_number}}</td>
            <td>{{this.course_name}}</td>
            <td>
              <a href="/courses/edit/{{this.id}}">Edit</a>
              <a href="/courses/delete/{{this.id}}">Delete</a>
            </td>
          </tr>
        {{/each}}
      </tbody>
    </table>
    <button class="btn btn-primary" onClick="addCourse();">Add</button>
  </div>
</div>
```

The following layout is used for all the views in the application.

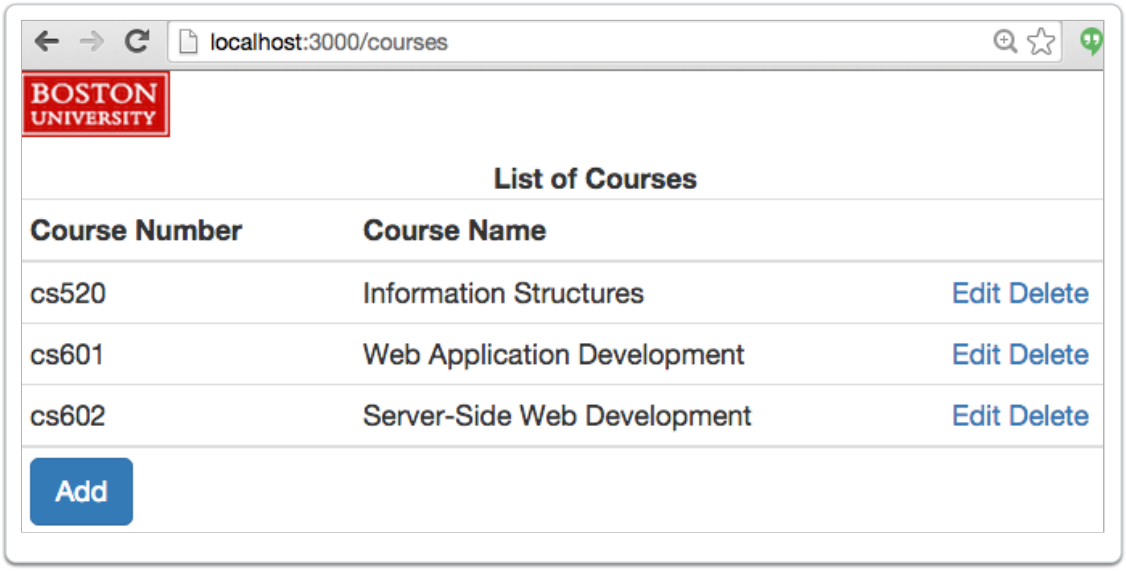
main_logo.handlebars

```
<!doctype html>
<html>
<head>
  <title>CS602</title>
  <link rel='stylesheet' href='/stylesheets/bootstrap.css' />
  <script src="/javascripts/clickActions.js"
    type="text/javascript"></script>
</head>
<body>
<header>
  <img src="/images/bu-logo.gif" alt="logo">
  <p>
  <center><b>{{title}}</b></center>
</header>

  {{{body}}}

</body>
</html>
```

The initial list of courses is now displayed in the browser as shown below.



The JavaScript file included in the web page has the actions for handling the button clicks as shown below.

clickActions.js

```
function addCourse(){
    window.location.href = '/courses/add';
}

function cancelAdd(){
    window.location.href = '/courses';
}
```

The *addCourse* module exports the function that renders the view for adding a new course.

addCourse.js

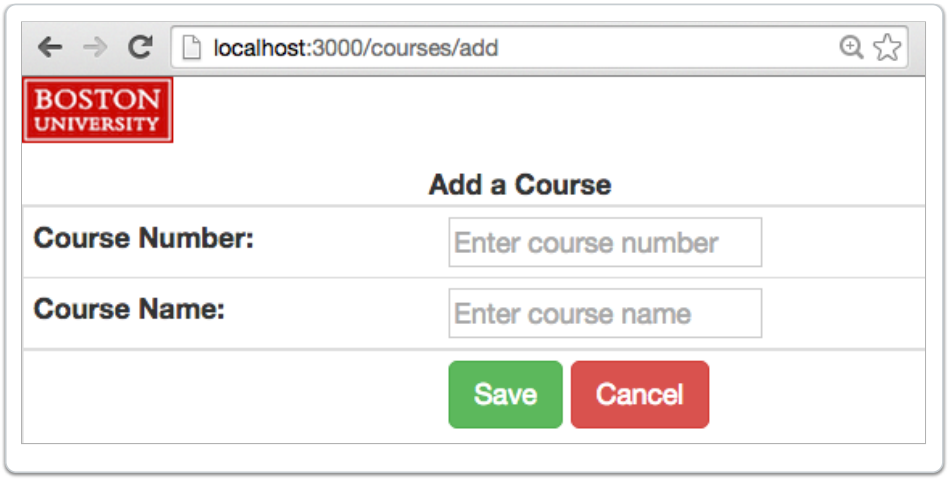
```
module.exports =
  function addCourse(req , res , next){
    res.render('addCourseView',
      {title:"Add a Course"});
};
```

The view page for adding a new course is shown below. In addition to the inputs for the course name and course number, buttons are provided for saving the course data or cancel the operation.

addCourseView.handlebars

```
<div class="table-responsive">
 <form method="post" action="/courses/add">
   <table class="table table-condensed">
     <tbody>
      <tr>
       <th>Course Number: </th>
       <td><input type="text" name="cnumber" required
            placeholder="Enter course number"></td>
      </tr>
      <tr>
        <th>Course Name: </th>
        <td><input type="text" name="cname" required
            placeholder="Enter course name"></td>
      </tr>
     </tbody>
     <tfooter>
       <tr>
         <td></td>
         <td>
          <input type="submit" value="Save"
            class="btn btn-success">
          <input type="button" value="Cancel"
            class="btn btn-danger" onclick="cancelAdd()">
         </td>
       </tr>
     </tfooter>
   </table>
 </form>
</div>
```

The initial layout of the view is shown below.



When the form is submitted, the *saveCourse* module exports the functionality for saving the data into the database. Since the form is submitted via POST, the course number and course name are obtained through the request's body.

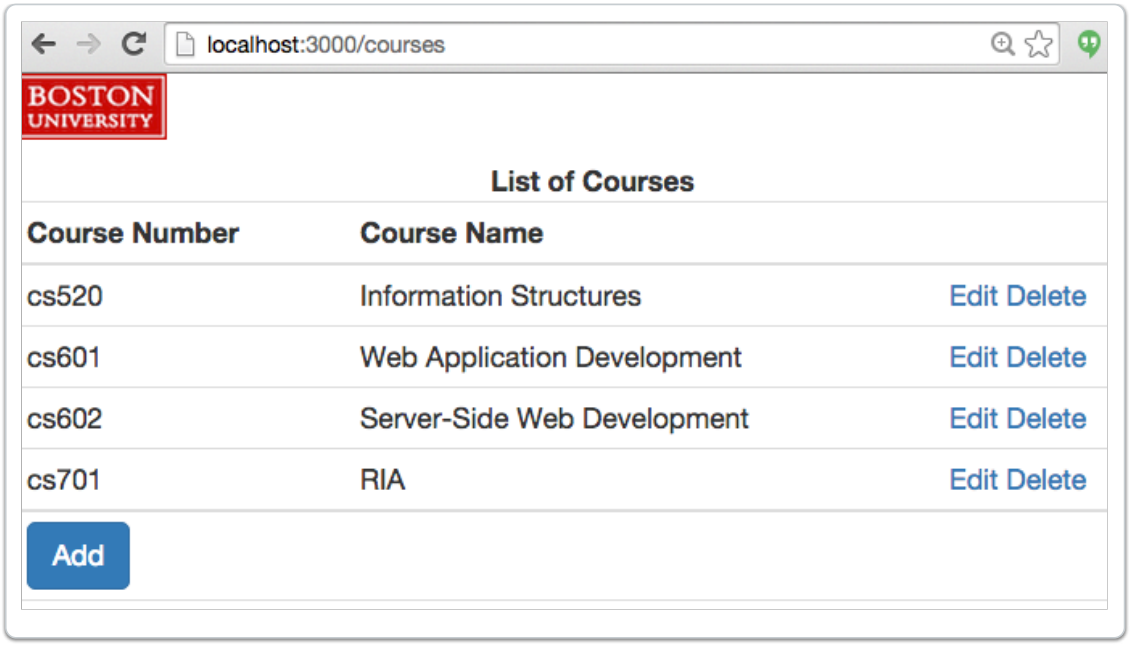
saveCourse.js

```
var connection =
  require('./dbConnection.js').dbConnect();

module.exports =
  function saveCourse(req , res , next){

    var inputFromForm = {
      course_number  : req.body.cnumber,
      course_name    : req.body.cname
    };
    connection.query("INSERT INTO met_courses set ? ",
      inputFromForm,
      function(err, rows)
      {
        if (err)
          console.log("Error inserting : %s ",err );
        res.redirect('/courses');
      });
  };
```

After the save is successful, the application redirects the user to the course list as shown below. The last row in the list shows the information about the newly added course.



The *editCourse* module exports the function that renders the view for editing an existing course with the specified *id* through the request's parameter. The first row of the retrieved data is passed to the view page as shown below.

editCourse.js

```
1  var connection =
2    require('./dbConnection.js').dbConnect();
3
4  module.exports =
5    function editCourse(req , res , next){
6      var id = req.params.id;
7      connection.query('SELECT * FROM met_courses WHERE id = ?',
8        [id],
9        function(err,rows){
10         if(err)
11           console.log("Error Selecting : %s ", err);
12         res.render('editCourseView',
13           {title:"Edit Course", data:rows[0]});
14     });
15 };
```

The view page for editing the selected course is shown below. In addition to the inputs for the course name and course number, buttons are provided for saving the course data or cancel the operation.

editCourseView.handlebars

```
<div class="table-responsive">
  <form method="post" action='/courses/edit/{{data.id}}'>
    <table class="table table-condensed">
      <tbody>
        <tr>
          <th>Course Number: </td>
          <td><input type="text" name="cnumber" required
                value="{{data.course_number}}"></td>
        </tr>
        <tr>
          <th>Course Name: </td>
          <td><input type="text" name="cname" required
                value="{{data.course_name}}"></td>
        </tr>
      </tbody>
      <tfooter>
        <tr>
          <td></td>
          <td>
              <input type="submit" value="Save" class="btn btn-success">
              <input type="button" value="Cancel" class="btn btn-danger"
                class="cancel" onclick="cancelAdd()">
          </td>
        </tr>
      </tfooter>
    </table>
  </form>
</div>
```

The layout for the web page while editing the course is shown below. The existing values are shown in the input fields.

.

When the user submits this form for the modifications, the *saveAfterEdit* module exports the required functionality as shown below. The course number and course name are updated for the specified *id*. The former values are obtained through the request's body while the latter is retrieved from the request's parameters.

.

After the update is successful, the application redirects the user to the course list as shown below.

.

The *deleteCourse* module exports the function that deletes the specified course whose *id* is specified in the request's parameters.

.

After the delete is successful, the application redirects the user to the course list as shown below.

.

References

Node MySQL, https://github.com/felixge/node-mysql

Persistence using MongoDB

Learning Objectives

By reading the lectures and textbook, participating in the discussions, and completing the assignments, you will be able to do the following:

- Write standalone MySQL database applications.
- Develop Node web applications that persist and interact with data form MySQL database.

Introduction

MongoDB is one of the popular open source NoSQL databases used along with Node.js. MongoDB is a document-oriented database and stores data in BSON (Binary JSON) format. Check out the instructions for installing MongoDB.

After installing MongoDB, run the mongod process located in the *bin* folder of the installation.

For the *Node* applications to interact with the MongoDB database, the *mongodb* module is required and is installed for the applications as shown below. This is the *Node.js* driver for MongoDB written in JavaScript.

Connecting to MongoDB

A connection is required for accessing the MongoDB database. The connection is created using the connect() method from the *mongodb* module's *MongoClient* property. The first argument for this method is the connection string that specifies the database, *cs602db,* to be used in MongoDB. The second argument is the callback function invoked after the connection is successful. The second argument of the callback function gives access to the database object if there is no error. The database is created on the fly in MongoDB if it doesn't exist already.

The following example attempts the connection to the database, prints the name of the database, and closes the connection.

The output of the above program on successful connection is shown below.

If the MongoDB instance is not running, the above program results in an error while trying to establish a connection, as shown below.

Document Collection

A document is the unit of storage in the MongoDB database. A document is analogous to a record in the relational database. A collection in MongoDB is associated with a store of the documents for that collection. A collection is analogous to a table in the relational database. Since MongoDB is a schema free database, documents of various structures may be stored in the same collection. A collection, if it doesn't exist, is automatically created once a document is inserted.

Inserting a Document

The insert() method adds a document to the specified collection. If the document doesn't contain the *_id* field, MongoDB will create the *_id* property for each document object and assigns a unique *ObjectId* before inserting the document into the collection. If the *_id* field already exists in the document, the value has to be unique in the collection in order to avoid a duplicate key error.

The following example accesses the *people* document collection and inserts a new object that has *firstName* and *lastName* as properties. On successful insertion, the callback function's second argument can be used to examine the number of inserted documents, and their *_id* values.



The output of the above program is shown below. The *newData* object now has the *_id* property in addition to the *firstName* and *lastName*, upon insertion.



Finding all Documents

The find() method selects documents in the specified collection. The method can take two parameters, both of which are optional. The first optional argument is the query specifying the selection criteria. For returning all documents in a collection, the empty document {} is passed as the first argument, or can be omitted completely. The second optional argument specifies the fields to be returned in the matching documents. If this argument is omitted, all the fields are included in the results. The method returns a cursor to the matching documents that can converted to an array as shown in the example below. The callback function's second argument receives the array of matched documents.



The output of the above program is shown below.

